

Library Management System

Aim: To understand and implement the core Object-Oriented Programming (OOP) concepts in Java.

Algorithm:

1. Start.
2. Create classes for Book, Person, Student, Faculty, Area, Vehicle, Car, Bike, Shape, Circle, Rectangle, and Report.
3. Implement constructors and encapsulation in Book.
4. Implement inheritance using Student and Faculty.
5. Implement method overloading using Area.
6. Implement method overriding using Car and Bike.
7. Implement abstraction using Shape.
8. Implement an interface using Printable and Report.
9. Create required objects in main().
10. Call the respective methods and display the results.
11. Stop.

Program Code:

```
// PART A and B: Classes, Objects, Constructors, Encapsulation  
class Book{  
    private int bookID;
```

```
private String bookname;  
private String author;  
private double price;
```

```
// Default constructor
```

```
Book() {  
    bookID = 0;  
    bookname = "Unknown";  
    author = "Unknown";  
    price = 0.0;  
}
```

```
// Parameterized constructor
```

```
Book(int bookID, String bookname, String author, double price) {  
    this.bookID = bookID;  
    this.bookname = bookname;  
    this.author = author;  
    this.price = price;  
}
```

```
// Getters and Setters
```

```
int getbookID() {  
    return bookID;
```

```
}
```

```
void setbookID(int bookID) {
```

```
    this.bookID = bookID;
```

```
}
```

```
String getbookname() {
```

```
    return bookname;
```

```
}
```

```
void setbookname(String bookname) {
```

```
    this.bookname = bookname;
```

```
}
```

```
String getauthor() {
```

```
    return author;
```

```
}
```

```
void setauthor(String author) {
```

```
    this.author = author;
```

```
}
```

```
double getprice() {
```

```
    return price;
```

```
}
```

```
void setprice(double price) {
```

```
this.price = price;  
}
```

```
// Display book details
```

```
void displayBook() {  
    System.out.println("Book ID  : " + bookID);  
    System.out.println("Book Name : " + bookname);  
    System.out.println("Author   : " + author);  
    System.out.println("Price    : " + price);  
}  
}
```

```
// PART C: INHERITANCE
```

```
class Person {  
    String name;  
    int age;  
  
    Person(String name, int age) {  
        this.name = name;  
        this.age = age;  
    }  
  
    void displayPerson() {
```

```
        System.out.println("Name : " + name);  
        System.out.println("Age : " + age);  
    }  
}
```

// Student inherits Person

```
class Student extends Person {  
    int rollNo;  
  
    Student(String name, int age, int rollNo) {  
        super(name, age);  
        this.rollNo = rollNo;  
    }  
}
```

```
void displayStudent() {  
    displayPerson();  
    System.out.println("Roll No : " + rollNo);  
}  
}
```

// Faculty inherits Person

```
class Faculty extends Person {  
    String subject;
```

```
Faculty(String name, int age, String subject) {  
    super(name, age);  
    this.subject = subject;  
}
```

```
void displayFaculty() {  
    displayPerson();  
    System.out.println("Subject : " + subject);  
}  
}
```

// PART D: POLYMORPHISM

// Method Overloading

```
class Area {
```

```
    // Area of square
```

```
    double calculateArea(double side) {  
        return side * side;  
    }
```

```
    // Area of rectangle
```

```
    double calculateArea(double length, double breadth) {
```

```
    return length * breadth;  
}
```

```
// Area of circle
```

```
double calculateArea(float radius) {  
    return 3.14 * radius * radius;  
}  
}
```

```
// Method Overriding
```

```
class Vehicle {  
    void display() {  
        System.out.println("This is a Vehicle");  
    }  
}
```

```
class Car extends Vehicle {  
    @Override  
    void display() {  
        System.out.println("This is a Car");  
    }  
}
```

```
class Bike extends Vehicle {  
    @Override  
    void display() {  
        System.out.println("This is a Bike");  
    }  
}
```

// PART E: ABSTRACTION

```
abstract class Shape {  
  
    // Abstract method  
    abstract void draw();  
}
```

```
class Circle extends Shape {  
  
    @Override  
    void draw() {  
        System.out.println("Drawing a Circle");  
    }  
}
```

```
class Rectangle extends Shape {
```



```
@Override
void draw() {
    System.out.println("Drawing a Rectangle");
}
}

// INTERFACE
interface Printable {
    void print();
}

class Report implements Printable {

    @Override
    public void print() {
        System.out.println("Printing Library Report");
    }
}

public class LibraryManagementSystem {
    public static void main(String[] args) {

        // PART A + B: Book Objects
```

```
System.out.println("===== BOOK DETAILS =====");
```

```
// Object using parameterized constructor
```

```
Book b1 = new Book(101, "Java Programming", "Santhosh", 500);
```

```
b1.displayBook();
```

```
System.out.println();
```

```
// Object using default constructor
```

```
Book b2 = new Book();
```

```
// Using setters
```

```
b2.setbookID(102);
```

```
b2.setbookname("Data Structures");
```

```
b2.setauthor("Ganga Parvathi");
```

```
b2.setprice(450);
```

```
b2.displayBook();
```

```
// Using getters
```

```
System.out.println("\nBook Name using Getter: "+  
b1.getbookname());
```

```
// PART C: Inheritance
```

```
System.out.println("\n===== INHERITANCE =====");
```

```
Student s = new Student("Avinash", 20, 106);
```

```
s.displayStudent();
```

```
System.out.println();
```

```
Faculty f = new Faculty("Dr.James", 40,"Computer Science");
```

```
f.displayFaculty();
```

```
// PART D: Method Overloading
```

```
System.out.println("\n===== METHOD OVERLOADING =====");
```

```
Area a = new Area();
```

```
System.out.println("Area of Square: "+ a.calculateArea(5.0));
```

```
System.out.println("Area of Rectangle: "+ a.calculateArea(5.0, 4.0));
```

```
System.out.println("Area of Circle: "+ a.calculateArea(3.0f));
```

```
// PART D: Method Overriding
```

```
System.out.println("\n===== METHOD OVERRIDING =====");
```

```
Vehicle v1 = new Car();
```

```
Vehicle v2 = new Bike();
```

```
v1.display();
```

```
v2.display();
```

```
// PART E: Abstraction
```

```
System.out.println("\n===== ABSTRACTION =====");
```

```
Shape c = new Circle();
```

```
Shape r = new Rectangle();
```

```
c.draw();
```

```
r.draw();
```

```
// PART E: Interface
```

```
System.out.println("\n===== INTERFACE =====");
```

```
Report = new Report();
```

```
report.print();
```

```
}
```

```
}
```

Output:

===== BOOK DETAILS =====

Book ID : 101

Book Name : Java Programming

Author : Santhosh

Price : 500.0

Book ID : 102

Book Name : Data Structures

Author : Ganga Parvathi

Price : 450.0

Book Name using Getter: Java Programming

===== INHERITANCE =====

Name : Avinash

Age : 20

Roll No : 106

Name : Dr. James

Age : 40

Subject : Computer Science

===== METHOD OVERLOADING =====

Area of Square: 25.0

Area of Rectangle: 20.0

Area of Circle: 28.26

===== METHOD OVERRIDING =====

This is a Car

This is a Bike

===== ABSTRACTION =====

Drawing a Circle

Drawing a Rectangle

===== INTERFACE =====

Printing Library Report

Theory Questions and Answers

1. What is Object-Oriented Programming?

Object-Oriented Programming (OOP) is a programming approach

An object contains:

- Data (attributes)
- Methods (behavior)

Java is mainly an object-oriented programming language.

Example:

```
class Student {  
    String name;  
    void display() {  
        System.out.println(name);  
    }  
}
```

2. Differentiate between a Class and an Object.

Class:

- A class is a blueprint or template for creating objects.
- It defines data members and methods.
- It does not represent a specific entity by itself.

Object:

- An object is an instance of a class.
- It represents a real entity.
- It occupies memory.

Example:

```
class Book { }
```

```
Book b1 = new Book();
```

Book is the class and b1 is the object.

3. Explain the purpose of constructors.

A constructor is a special method-like member used to initialize

Important points:

- It has the same name as the class.
- It has no return type.
- It is automatically called when an object is created.
- It initializes instance variables.

Types commonly used:

1. Default constructor
2. Parameterized constructor

Example:

```
Book() {
```

```
price = 0.0;
```

```
}
```

```
Book(int id, double p) {
```

```
bookID = id;
```

```
price = p;  
}
```

4. What is Encapsulation? How is it implemented in Java?

Encapsulation means wrapping data and methods into a single

It is commonly implemented using:

- private data members
- public/proper getter and setter methods

Example:

```
class Book {  
    private double price;  
    double getprice() {  
        return price;  
    }  
    void setprice(double price) {  
        this.price = price;  
    }  
}
```

Here price cannot be accessed directly from outside the class.

5. Differentiate between Method Overloading and Method Overriding.

Method Overloading:

- Same method name
- Different parameter list
- Usually occurs in the same class
- Compile-time polymorphism

Example:


```
calculateArea(double side)
calculateArea(double length, double breadth)
```

Method Overriding:

- Same method name
- Same parameter list
- Child class provides its own implementation
- Runtime polymorphism

Example:

```
class Car extends Vehicle {
void display() {
System.out.println("This is a Car");
}
}
```

6. Explain the four pillars of Object-Oriented Programming with suitable examples.

The four pillars of OOP are:

1. Encapsulation

Wrapping data and methods together and controlling access.

Example: private variables with getters/setters.

2. Inheritance

One class acquires properties and methods of another class.

Example:

```
class Student extends Person { }
```

3. Polymorphism

One name can have different forms.

Examples:

- Method overloading
- Method overriding

4. Abstraction

Hiding implementation details and showing only essential features

Example:

```
abstract class Shape {  
    abstract void draw();  
}
```

In the given program:

- Encapsulation → Book
- Inheritance → Student and Faculty
- Polymorphism → Area, Car and Bike
- Abstraction → Shape, Circle and Rectangle

7. Differentiate between Abstract Classes and Interfaces.

Abstract Class:

- Declared using the abstract keyword.
- Can contain abstract and concrete methods.
- Can have constructors and instance variables.
- A class extends an abstract class.

Interface:

- Declared using the interface keyword.
- Used to define a contract that implementing classes follow.
- A class implements an interface.
- A class can implement multiple interfaces.

Example:

```
abstract class Shape {  
    abstract void draw();  
}  
  
interface Printable {  
    void print();  
}
```

```
class Report implements Printable {  
    public void print() {  
        System.out.println("Printing Library Report");  
    }  
}
```